Phenix Documentation — Miscellaneous

6 documentation pages
Generated from local Phenix documentation

Contents

AI automation of structure determination

ai_agent.html

Authors

- Tom Terwilliger, Billy Poon, Pavel Afonine, Dorothee Liebschner, Peter Zwart, Airlie McCoy

Purpose

The Phenix AI Agent automates macromolecular structure determination. You give it your experimental data and (optionally) some guidance, and it figures out which PHENIX programs to run, in what order, with what settings — and then runs them for you automatically.

Think of it as an experienced crystallographer sitting next to you at the computer. It looks at your data, decides what to do first, checks the results, and decides what to do next. If something goes wrong, it tries a different approach. When it's done, it tells you what it found and where to look.

Quickest way to try it: run a tutorial

The fastest way to see the AI Agent in action is to run one of the built-in PHENIX tutorials:

- In the PHENIX GUI, go to File → Tutorials
- Pick a tutorial (e.g. "p9-sad" for Se-SAD phasing, or any MR tutorial with diffraction data)
- Open the AI Agent from the main program list
- The GUI will detect the tutorial automatically — you'll see a blue banner with the tutorial name

All you need to do is click Run. The agent reads the tutorial's README file, extracts the experiment parameters (wavelength, atom type, resolution, etc.), selects a plan template, and runs the entire workflow automatically. No advice, no settings changes, no file selection needed.

The p9-sad tutorial, for example, completes in 5 cycles with 0 failures, producing a model with R-free 0.247 at 1.74 Å resolution.

How the AI Agent works

The AI Agent expects to be given data files from crystallography (mtz files) or from cryo-EM (mrc or ccp4 maps), a sequence file, and optional model files or ligand files. You can tell it what files to use by specifying a directory containing the files or by loading the files in the AI Agent GUI window.

The AI Agent also accepts instructions. You can say things like "first run xtriage and then molecular replacement", or "solve the structure", or "use main.number_of_macro_cycles=1 in phenix refine", or whatever else you want to tell it to do with your X-ray or cryo-EM data.

Note on instructions: if you are supplying PHIL keywords for the agent, specify them exactly as you would on the command line. For example:

```
use main.number_of_macro_cycles=1 in refinement
```

but not:

```
use one macro_cycle in refinement
```

(macro_cycles happens to be a special case that the agent recognizes, but most parameters are not converted like this.)

You can supply instructions in the GUI. If your directory contains a README file the agent can read it and follow the instructions in it.

Structure solution pathways. The AI Agent selects from 17 plan templates covering X-ray MR, SAD, MAD, cryo-EM, ligand fitting, and other workflows. A typical pathway for X-ray data might be:

```
xtriage → autosol → autobuild → refine → molprobity
```

if the data have anomalous signal, or:

```
xtriage → predict_and_build → refine
```

if they do not. For cryo-EM data:

```
mtriage → resolve_cryo_em → dock_in_map → real_space_refine
```

If you supply a ligand, the pathway will end with ligand fitting and model refinement. You can modify most aspects of the pathway with your instructions, as long as the necessary data is available.

How decisions are made. Each cycle, the agent runs a six-node decision pipeline: PERCEIVE (extract metrics, categorize files) → THINK (analyze logs with crystallographic expertise) → PLAN (select next program) → BUILD (construct the command) → VALIDATE (check workflow rules) → OUTPUT (update session). The LLM participates in two nodes (THINK and PLAN); the other four are deterministic. The LLM sets the intent; deterministic code enforces the accuracy.

In Expert mode (the default), the agent also creates a multi-stage strategy plan at the start of the session. After each cycle, a deterministic gate compares metrics to the stage's success criteria and decides whether to advance, retreat, or stop.

What the LLM does not control. The LLM cannot write arbitrary command-line strings. It sets strategy flags (e.g. atom_type=Se, resolution=2.5) which the BUILD node expands through programs.yaml into validated PHIL parameters. File paths, resolution values, R-free flags, and output prefixes are all injected by deterministic code. Any parameter not in the program's allowlist is stripped by the command sanitizer.

What the agent shows you while it runs

Once you click Run, the PHENIX GUI switches to the Agent Progress tab. Here is what you'll see, illustrated with real output from the p9-sad tutorial.

The plan. In Expert mode, the first thing you see is the strategy plan:

```
================================================
 STRATEGY PLAN: SAD/MAD experimental phasing (X-ray)
================================================
 ■ Stage 1: Analyze data quality and anomalous signal
 Goal: Data quality analysis complete
 ■ Stage 2: SAD/MAD phasing and initial model building
 Goal: Experimental phasing complete
 ■ Stage 3: Rebuild and refine model
 Goal: R-free &lt;0.30
 ■ Stage 4: Final refinement with ordered solvent
 Goal: R-free &lt;0.25
 ■ Stage 5: Final model validation
================================================
```

Each stage has a goal. The symbols show status: ■ pending, ● active, ✓ complete, ■ skipped, ✗ failed.

Cycle output. Each program the agent runs appears as a numbered cycle with its decision, reasoning, and result. Here is cycle 2 (autosol) from the same run:

```
Cycle 2: phenix.autosol

Decision: phenix.autosol
 Reasoning: User requested SeMet SAD with additional
 sulfur search using AutoSol, truncating to 2.5 Å,
 wavelength 0.9792 Å, and ~5 Se sites. Xtriage
 indicates usable anomalous signal, so experimental
 phasing is appropriate.
 Source: llm (openai)

File Selection:
 Data_Mtz: p9.sca (Reason: llm_selected)
 Sequence: seq.dat (Reason: llm_selected)

Command:
 phenix.autosol autosol.data=p9.sca seq_file=seq.dat
 autosol.lambda=0.9792 resolution=2.5
 autosol.atom_type=Se mad_ha_add_list=S
 autosol.sites=5 nproc=4

Running: phenix.autosol ... [OK]

[GATE] Phase complete: experimental_phasing →
 build_and_refine
```

The Reasoning field shows exactly why the agent made this choice. The Source field tells you whether the LLM or the rules engine made the decision. The GATE line shows the plan advancing to the next stage.

Stage transitions. When the agent completes or retreats from a stage:

```
✓ STAGE COMPLETE: Analyze data quality
 All steps completed (1/1 cycles) — advancing

■ RETREAT: initial_refinement → molecular_replacement
 R-free stuck above 0.45 after 3 cycles
```

A retreat is not a failure — it means the agent recognized that its current approach isn't working and is trying something different.

What success looks like

Here is the complete output from the p9-sad tutorial (5 cycles, 0 failures):

```
Cycle 1: phenix.xtriage → Data quality OK
 Resolution: 1.74 Å, Space group: I 4
 Anomalous measurability: 0.198

Cycle 2: phenix.autosol → SAD phasing succeeds

Cycle 3: phenix.autobuild → R-free: 0.230

Cycle 4: phenix.molprobity → Clashscore: 12.46

Cycle 5: STOP
 R-free = 0.2466 (&lt; 0.25 target). Stopping.

Final Quality:
 R-free 0.2466 Good
 R-work 0.2295
 Clashscore 12.5 Acceptable
 Rama Outliers 2.4%
 Rotamer Outliers 1.9%
```

```
Key Output Files:
 overall_best_final_refine_001.pdb (model)
 overall_best_refine_data.mtz (data)
 overall_best_refine_map_coeffs.mtz (map coefficients)
```

Understanding the metrics

If R-free is going down cycle by cycle, things are working.

Output files

All output files are in the agent's working directory (usually ai_agent_directory/ inside your project folder).

Your refined model: The .pdb file from the last successful refinement cycle. The Results panel in the GUI shows you exactly where each output file is.

Map coefficients: The .mtz file from the last refinement, containing 2Fo-Fc and Fo-Fc maps. Open this in Coot alongside the refined model to inspect the electron density.

HTML structure report: structure_report.html — a self-contained report with final metrics, an R-free trajectory chart, a workflow timeline, and output file locations. Click the Open Structure Report button in the Results tab to view it.

Text structure report: structure_determination_report.txt — a human-readable summary of the entire determination.

Session summary: session_summary.json — a machine-readable summary with final metrics, stage outcomes, and metric trajectory. Useful for comparing multiple runs programmatically.

What to do with the results: the Coot checklist

The agent produces a model, but you are the scientist. Before depositing or publishing, always inspect the model manually:

- Inspect the difference density map. Display the Fo-Fc map (green/red). Large positive peaks (green) may indicate missing atoms, alternate conformations, or unmodeled ligands.
- Check Ramachandran outliers. In Coot, go to Validate → Ramachandran Plot. Decide whether the density supports each unusual backbone conformation.
- Verify ligand placement. If the agent fit a ligand, examine it in the density. If the real-space correlation (RSCC) is below 0.7, inspect closely.
- Look at B-factor distribution. Regions with very high B-factors may be poorly ordered or incorrectly built.
- Check crystal contacts. Use Coot's symmetry display to verify sensible crystal contacts and correct handling of special positions.

When the AI Agent cannot resolve a problem

If the AI Agent encounters a fatal error (such as a crystal symmetry mismatch between your files, or a missing SHELX installation), it stops the run and produces an HTML diagnosis page that describes the problem and what you can do to fix it. The diagnosis contains three sections: "What went wrong," "Most likely cause," and "How to fix it."

If the agent is unable to produce an LLM-generated diagnosis (e.g. in rules-only mode), a simpler deterministic diagnosis is produced instead.

Common problems and what to do

"R-free stuck above 0.40." The model isn't fitting the data. Check the xtriage output for warnings about space group or twinning. Try a different search model or let the agent use AlphaFold (provide only the sequence).

"Trying a different approach" (RETREAT). The agent recognized that its current strategy isn't working and is backtracking. This is expected behavior — not a failure.

"No matching plan template." The agent couldn't find a strategy for your combination of files and advice. It will still run in reactive mode (choosing programs one at a time). Check that you've provided the right files for your experiment type.

"Safety Stop" or "Agent stopped." Something is fundamentally wrong that the agent cannot fix by trying different programs. The stop message will explain the issue. Check the xtriage output, verify your input files, and look at the failure diagnosis if one was produced.

Settings reference

Analysis Depth (thinking_level). The most important setting:

For most runs, Expert (the default) is the best choice.

Max cycles. Maximum programs to run (default: 20). A typical structure determination takes 5–15 cycles.

Restart mode. Fresh (start new) or Resume (continue from where the previous run left off).

Provider. Which AI to use: Google (default), OpenAI, or Ollama (local).

Verbosity. Quiet (errors only), Normal (decisions and metrics), or Verbose (full detail including file selection).

Job control with the AI Agent

If you have already run an AI Agent job, you can restore it in the GUI by clicking on Job History and then clicking on the run. At the bottom of the Configure panel, you can set:

- Restart mode: Fresh (start a new job) or Resume (keep going)
- Display Session and stop: None, Basic, or Detailed — shows what the agent has done so far without running anything
- Remove last N cycles and stop: Enter a number to roll back that many cycles (e.g. 5 to remove the last 5 decisions)

You can change the advice when you resume. The agent will try to follow your new instructions.

Limitations of the AI Agent

- The agent processes program logs and numerical metrics. It does not see density maps directly — spatial information (ligand shape, disorder) that an experienced crystallographer would see at a glance is not available to the agent.
- One dataset per session. No multi-crystal merging, serial crystallography, or ensemble strategies.
- 23 PHENIX programs are currently registered. Some programs (ensemble_refinement, local sharpening) are not yet available.
- The LLMs used in choosing programs have varying capabilities and can sometimes suggest steps that might not be the best choices.

- AI tools can make mistakes. Use the agent as a helper; don't expect it to always be right. Always inspect the model in Coot.

Privacy

If you use the AI Agent, your log files are sent to the Phenix server, and from there to OpenAI or Google Gemini. That means the data could potentially be used by those providers in any way that they use other data sent to them.

Running locally with Ollama keeps all data on your machine. See below for setup instructions.

Note that access to OpenAI and Gemini through Phenix is non-commercial only.

Running with the Phenix server

Normally you will use Google (Gemini) as the LLM provider. This requires an API key for Google (supplied with Phenix). You can also use Ollama (run on the Phenix server) or OpenAI (also uses a supplied API key).

A shared set of keys is supplied, so you can run without getting your own key. The number of analyses with shared keys is limited, however.

Running locally

You can run on your own machine (not using the Phenix server) by unchecking "Run on server" in the GUI or setting run_on_server=False on the command line.

Your options when running locally:

- Ollama — fully local, requires your own Ollama installation
- Google — your machine calls Google's API (needs your API key)
- OpenAI — your machine calls OpenAI's API (needs your API key)

Running locally with Ollama

If you install Ollama on your machine (typically with a GPU), you can run everything locally without sending any data outside your machine.

After installing Ollama, set up the required models:

```
ollama pull llama3.1:70b
ollama pull llama3.1:8b
ollama pull nomic-embed-text
```

Set the environment variables:

```
setenv CUDA_VISIBLE_DEVICES 0,1,2
setenv OLLAMA_NUM_PARALLEL 6
setenv OLLAMA_HOST 0.0.0.0:11434
```

Then run the agent with:

```
run_on_server=False
provider=ollama
```

Note: the required models may change with Phenix versions.

Running with an API key

If you have a Google API key (set GOOGLE_API_KEY) or an OpenAI API key (set OPENAI_API_KEY), you can run locally with:

```
run_on_server=False
provider=google (or provider=openai)
```

You can test your API keys with:

```
phenix.python $PHENIX/modules/cctbx_project/libtbx/langchain/tests/test_api_keys.py
```

(Note: the path to this test could change.)

Setting up a Google API key

Getting a Google API key requires several steps:

- Set up a Google account: Go to gmail.com and create an account.
- Activate billing: Log in, go to https://console.cloud.google.com/billing and click Activate. You need a credit card but it won't be charged unless you convert to a paid account.
- Create an API key: Go to https://console.cloud.google.com/apis/credentials and create a new credential. Copy the API key.
- Enable the API: Go to https://console.cloud.google.com/apis/library, search for "Generative Language API", select it, and click ENABLE.
- Restrict the key: Go back to your credentials, click your key. Under "Application Restrictions" select "IP Addresses" and enter your IP addresses. Under "API restrictions" select "Restrict key" and choose "Generative Language API".
- Set the environment variable: setenv GOOGLE_API_KEY xxxxxxxx
- In the GUI, uncheck "Run on server" and set provider=google.

Google gives you credits good for 90 days. Beyond that you pay for access.

Setting up an OpenAI API key

Setting up an OpenAI key is simpler but there is no free trial:

- Log into openai.com (or create an account).
- In the left menu, click "API keys".
- Click "+ Create new secret key". Give it a name.
- Copy the key immediately (it's shown only once). Store it securely.
- setenv OPENAI_API_KEY sk-xxxxxxxx
- In the GUI, uncheck "Run on server" and set provider=openai.
- You pay for access from the start.

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True log_directory = None job_id = None history_file = None history_simple_string =

None max_history = 10 iterate_agent = True max_cycles = 20 restart_mode = *fresh resume Fresh: start a new analysis from scratch. Resume: continue from the previous run's session.auto_recovery = True Automatically attempt to recover from certain structured errors (e.g., ambiguous data labels in MTZ files). Set to False for debugging or when you want full manual control.project_advice = None input_directory = None Directory containing input files and optional README with instructions. If a README file is found, its contents will be used as additional advice for the agent.preprocess_advice = True Use LLM to preprocess and clarify user advice into structured format. This helps the agent better understand informal or brief instructions.readme_file_patterns = README README.txt README.dat README.md notes.txt Filenames to look for when extracting advice from input_directory. Search is case-insensitive.max_readme_chars = 5000 Truncate README files longer than this to avoid excessive LLM input.maximum_automation = True use_rules_only = False When True, the agent skips ALL LLM calls: - Program selection uses deterministic rules from YAML config - Directive extraction uses simple pattern matching - Advice preprocessing returns raw advice unchanged - Session summaries are skipped This runs completely offline without external API calls. Useful for testing or when API quota is exhausted.use_llm = True When False, equivalent to use_rules_only=True. The agent runs without any LLM calls, using only deterministic workflow rules for program selection. Set to False for fully offline operation.thinking_level = none basic advanced *expert Controls the depth of expert crystallographer reasoning applied at each decision point. none: No expert reasoning — original agent behavior. basic: Adds an LLM reasoning call with log analysis and strategy memory tracking. advanced (default): Full pipeline including structural validation, file metadata tracking, and expert knowledge base lookup. Increases LLM cost per cycle but substantially improves decision quality. expert: Adds goal-directed planning with multi-stage strategy, gate evaluation, retreat logic, hypothesis testing, and session reports. Requires advanced mode features (structural validation, structure model) as prerequisites.abort_on_red_flags = True When True, the agent will stop if it detects critical issues like experiment type changes or impossible workflow states. Set to False to continue despite red flags (not recommended).abort_on_warnings = False When True, the agent will also stop on warning-level issues like R-free spikes or missing resolution. Default is False (warnings are logged but don't stop execution).verbosity = quiet *normal verbose Controls how much detail is shown in the agent output. quiet: Only errors and final result. normal: Key decisions, metrics, and reasoning (default). verbose: Full detail including file selection and internal state.dry_run = False For testing: substitutes pre-recorded logs and outputs instead of running actual PHENIX programs. Use with use_rules_only=True for fully offline testing.dry_run_scenario = "xray_basic" Which test scenario to use for dry_run mode. Scenario folders are in tests/scenarios/.summarize_only = False Skip running any cycles and just generate a summary of the existing session. Useful for re-generating summaries or testing the summary functionality.include_llm_assessment = True When True, the final session summary will include an LLM-generated assessment...

- job_title = None Job title in PHENIX GUI, not used on command line
- ai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True log_directory = None job_id = None history_file = None history_simple_string = None max_history = 10 iterate_agent = True max_cycles = 20 restart_mode = *fresh resume Fresh: start a new analysis from scratch. Resume: continue from the previous run's session.auto_recovery = True Automatically attempt to recover from certain structured errors (e.g., ambiguous data labels in MTZ files). Set to False for debugging or when you want full manual control.project_advice = None input_directory = None Directory containing input files and optional README with instructions. If a README file is found, its contents will be used as additional advice for the agent.preprocess_advice = True Use LLM to preprocess and clarify user advice into structured format. This helps the agent better understand informal or brief

instructions.readme_file_patterns = README README.txt README.dat README.md notes.txt Filenames to look for when extracting advice from input_directory. Search is case-insensitive.max_readme_chars = 5000 Truncate README files longer than this to avoid excessive LLM input.maximum_automation = True use_rules_only = False When True, the agent skips ALL LLM calls: - Program selection uses deterministic rules from YAML config - Directive extraction uses simple pattern matching - Advice preprocessing returns raw advice unchanged - Session summaries are skipped This runs completely offline without external API calls. Useful for testing or when API quota is exhausted.use_llm = True When False, equivalent to use_rules_only=True. The agent runs without any LLM calls, using only deterministic workflow rules for program selection. Set to False for fully offline operation.thinking_level = none basic advanced *expert Controls the depth of expert crystallographer reasoning applied at each decision point. none: No expert reasoning — original agent behavior. basic: Adds an LLM reasoning call with log analysis and strategy memory tracking. advanced (default): Full pipeline including structural validation, file metadata tracking, and expert knowledge base lookup. Increases LLM cost per cycle but substantially improves decision quality. expert: Adds goal-directed planning with multi-stage strategy, gate evaluation, retreat logic, hypothesis testing, and session reports. Requires advanced mode features (structural validation, structure model) as prerequisites.abort_on_red_flags = True When True, the agent will stop if it detects critical issues like experiment type changes or impossible workflow states. Set to False to continue despite red flags (not recommended).abort_on_warnings = False When True, the agent will also stop on warning-level issues like R-free spikes or missing resolution. Default is False (warnings are logged but don't stop execution).verbosity = quiet *normal verbose Controls how much detail is shown in the agent output. quiet: Only errors and final result. normal: Key decisions, metrics, and reasoning (default). verbose: Full detail including file selection and internal state.dry_run = False For testing: substitutes pre-recorded logs and outputs instead of running actual PHENIX programs. Use with use_rules_only=True for fully offline testing.dry_run_scenario = "xray_basic" Which test scenario to use for dry_run mode. Scenario folders are in tests/scenarios/.summarize_only = False Skip running any cycles and just generate a summary of the existing session. Useful for re-generating summaries or testing the summary functionality.include_llm_assessment = True When True, the final session summary will include an LLM-generated assessment of the workflow. When False, only the structured summary is gener...

- log_file = None
- log_as_simple_string = None
- file_list_as_simple_string = None
- display_text_as_simple_string = None
- summary_as_simple_string = None
- analysis_as_simple_string = None
- load_existing_analysis = True
- program_name = None
- analysis_file_name = None
- summary_file_name = None
- write_files = True
- timeout = 180
- display_results = True
- log_directory = None
- job_id = None

• history_file = None

• history_simple_string = None

• max_history = 10

• iterate_agent = True

• max_cycles = 20

• restart_mode = *fresh resume Fresh: start a new analysis from scratch. Resume: continue from the previous run's session.

• auto_recovery = True Automatically attempt to recover from certain structured errors (e.g., ambiguous data labels in MTZ files). Set to False for debugging or when you want full manual control.

• project_advice = None

• input_directory = None Directory containing input files and optional README with instructions. If a README file is found, its contents will be used as additional advice for the agent.

• preprocess_advice = True Use LLM to preprocess and clarify user advice into structured format. This helps the agent better understand informal or brief instructions.

• readme_file_patterns = README README.txt README.dat README.md notes.txt Filenames to look for when extracting advice from input_directory. Search is case-insensitive.

• max_readme_chars = 5000 Truncate README files longer than this to avoid excessive LLM input.

• maximum_automation = True

• use_rules_only = False When True, the agent skips ALL LLM calls: - Program selection uses deterministic rules from YAML config - Directive extraction uses simple pattern matching - Advice preprocessing returns raw advice unchanged - Session summaries are skipped This runs completely offline without external API calls. Useful for testing or when API quota is exhausted.

• use_llm = True When False, equivalent to use_rules_only=True. The agent runs without any LLM calls, using only deterministic workflow rules for program selection. Set to False for fully offline operation.

• thinking_level = none basic advanced *expert Controls the depth of expert crystallographer reasoning applied at each decision point. none: No expert reasoning — original agent behavior. basic: Adds an LLM reasoning call with log analysis and strategy memory tracking. advanced (default): Full pipeline including structural validation, file metadata tracking, and expert knowledge base lookup. Increases LLM cost per cycle but substantially improves decision quality. expert: Adds goal-directed planning with multi-stage strategy, gate evaluation, retreat logic, hypothesis testing, and session reports. Requires advanced mode features (structural validation, structure model) as prerequisites.

• abort_on_red_flags = True When True, the agent will stop if it detects critical issues like experiment type changes or impossible workflow states. Set to False to continue despite red flags (not recommended).

• abort_on_warnings = False When True, the agent will also stop on warning-level issues like R-free spikes or missing resolution. Default is False (warnings are logged but don't stop execution).

• verbosity = quiet *normal verbose Controls how much detail is shown in the agent output. quiet: Only errors and final result. normal: Key decisions, metrics, and reasoning (default). verbose: Full detail including file selection and internal state.

• dry_run = False For testing: substitutes pre-recorded logs and outputs instead of running actual PHENIX programs. Use with use_rules_only=True for fully offline testing.

• dry_run_scenario = "xray_basic" Which test scenario to use for dry_run mode. Scenario folders are in tests/scenarios/.

• summarize_only = False Skip running any cycles and just generate a summary of the existing session. Useful for re-generating summaries or testing the summary functionality.

• include_llm_assessment = True When True, the final session summary will include an LLM-generated assessment of the workflow. When False, only the structured summary is generated (faster, no API call needed).

• analysis_mode = *standard agent_session advice_preprocessing Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), or advice_preprocessing (process user advice into structured format)

• session_json = None For agent_session mode, the session data as JSON string

• raw_advice = None For advice_preprocessing mode, the combined raw advice string

• experiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)

• file_list_hint = None For advice_preprocessing mode, comma-separated input file names

• user_advice_for_directives = None For directive_extraction mode, the processed user advice string

• original_files = None

• project_state_json = None

• resolution = None

• api_version = None When set, indicates request is in JSON format (v2.0)

• request_json = None Full JSON request containing files, history, settings, etc. Used when api_version is set.

• gui_mode = False When True, run sub-jobs via the program's native PHENIX launcher (template_launcher/run_program) for proper .pkl and .eff output, and send callbacks to the GUI for sub-job registration and live window opening. Falls back to easy_run if native launcher is not available. Set automatically by the GUI launcher.

• auto_open_sub_jobs = True

• display_and_stop = *None basic detailed Show the current session history and exit without running any cycles. basic: compact summary (program, result, R-free per cycle). detailed: full detail including files, metrics, and reasoning.

• remove_last_n = None Remove the last N cycles from the session and exit. Useful to roll back failed or unwanted cycles before resuming.

• communicationrun_on_server = True Run job on Phenix serverwait_for_server = False If server is busy or down, wait up to max_wait_timeupdate_wait_time_if_down = 5 Time to wait before trying again on server if it is downupdate_wait_time_if_busy = 5 Time to wait before trying again on server if it is busymax_wait_time = 450 Maximum wait timemax_server_tries = 1 Maximum calls to serverstop_if_internet_not_available = True Stop if attempting to predict models online and internet is not availablecycle = 1 Marker that can be used to identify cycleverbose = False Verbose output on communicationsprovider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• run_on_server = True Run job on Phenix server

• wait_for_server = False If server is busy or down, wait up to max_wait_time

• update_wait_time_if_down = 5 Time to wait before trying again on server if it is down

• update_wait_time_if_busy = 5 Time to wait before trying again on server if it is busy

• max_wait_time = 450 Maximum wait time

• max_server_tries = 1 Maximum calls to server

• stop_if_internet_not_available = True Stop if attempting to predict models online and internet is not available

• cycle = 1 Marker that can be used to identify cycle

• verbose = False Verbose output on communications

• provider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = prediction *ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 5 Time in seconds between job status checksmax_tries = 20 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = *small medium large Size of job (small, medium, large)requires_gpu = False Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose output

• url = None The URL for the Phenix REST server Normally set automatically

• url_type = prediction *ai Type of server url (prediction or ai). Normally set automatically

• port = None The port for interacting with the Phenix REST server Normally set automatically

• token = None Authentication token for accessing the Phenix REST server. Normally set automatically

• timeout = 5 Time to keep trying to connect to a server

• quick_check_interval = 1 Time in seconds between job status checks

• check_interval = 5 Time in seconds between job status checks

• max_tries = 20 Number of tries to get result from server

• max_tries_on_availability = 2 Number of tries to see if server is up

• stop_if_server_not_available = True Stop if server is not available (wrong url or down)

• job_size = *small medium large Size of job (small, medium, large)

• requires_gpu = False Job requires GPU

• running_server_test = False This is a test job

• verbose = False Verbose output

• guiGUI-specific parameter required for output directoryoutput_dir = None

• output_dir = None

AI analysis of Phenix results

ai_analysis.html

Authors

- Nigel Moriarty, Peter Zwart, Thomas C Terwilliger

Purpose

The Phenix AI analysis tool is designed to help you interpret your output files from a run in the Phenix GUI. It analyzes your log file and any output written to the GUI and the names of any output files that are supplied to the GUI and creates a summary of the run. Then it analyzes this summary in the context of the Phenix documentation to describe how this run fits into the framework of structure determination and suggests next steps. The AI analysis tool can be accessed in the Phenix GUI in the Results section of most Phenix GUI runs, next to the Log file button. You can also access it under the Tools menu.

Google and OpenAI API Keys

You can use Ollama (run on the Phenix server) to run the AI analysis, or you can use Google (gemini) or OpenAI. If you use Google or OpenAI, API keys for these servers are used. You can run AI analyses with Google or OpenAI without getting your own key, as a shared set of keys is supplied. The number of analyses with these shared keys is limited, however.

How the AI analysis works

The AI analysis is carried out in two steps. In the first step, the log file, along with text written to the GUI and names of output files supplied to the GUI are summarized using a Langchain analysis with reranking using FlashRank. In the second step, the summary from the first step is analyzed in the context of all the Phenix documentation, transcripts of Phenix tutorial videos, and Phenix publications to provide background and to suggest next steps to take.

This type of AI does not save or learn from your questions. However the information in your log file is sent to the Phenix server, and from there, on to Google gemini.

What to do with the AI analysis

The purpose of the AI analysis is to help you interpret your output and to suggest ideas for next steps. You should keep in mind that these tools can make mistakes and give you incorrect interpretations and poor suggestions at times, so you want to just take the information as suggestions to think about.

You can combine this AI analysis with the Phenix chatbot . The chatbot can give you interactive answers to your questions using the same database of information as the AI analysis. This allows you to follow up on the AI analysis with questions to the chatbot. You can also paste part of the output from the AI analysis into the chatbot along with a question to get more context.

Limitations of the AI analysis

The AI analysis is limited to the sources that is supplied with, so it only knows about Phenix, and it only knows what is in the documentation, the videos and newsletters, and the papers we have supplied.

AI tools like this one can also just make mistakes and give incorrect answers. This does not seem to happen too often with this tool, but you need to always be on alert when using it. Use the tool as a helper, don't expect it to always be right.

If a detail is missing in the documentation, the AI analysis will not know it.

Privacy in the AI analysis

If you use the AI analysis tool, your log files are sent to the Phenix server, and from there on to OpenAI and Gemini. That means the data could potentially be used by OpenAI, and Google in any way that they use other AI data that is sent to them.

Note that access to OpenAI and Gemini is non-commerical only.

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True analysis_mode = *standard agent_session advice_preprocessing directive_extraction failure_diagnosis Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), advice_preprocessing (process user advice into structured format), directive_extraction (extract structured directives from advice), or failure_diagnosis (LLM diagnosis of a diagnosable-terminal error)include_llm_assessment = True For agent_session mode, whether to include LLM-generated assessmentsession_json = None For agent_session mode, the session data as JSON stringraw_advice = None For advice_preprocessing mode, the combined raw advice stringexperiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)file_list_hint = None For advice_preprocessing mode, comma-separated input file namesuser_advice_for_directives = None For directive_extraction mode, the processed user advice stringfailure_error_type = None For failure_diagnosis mode: error type key from diagnosable_errors.yamlfailure_error_text = None For failure_diagnosis mode: encoded error excerpt from the failing programfailure_program = None For failure_diagnosis mode: the PHENIX program that failedfailure_log_tail = None For failure_diagnosis mode: encoded last section of the failing program logcommunicationrun_on_server = True Run job on Phenix serverwait_for_server = False If server is busy or down, wait up to max_wait_timeupdate_wait_time_if_down = 5 Time to wait before trying again on server if it is downupdate_wait_time_if_busy = 5 Time to wait before trying again on server if it is busymax_wait_time = 450 Maximum wait timemax_server_tries = 1 Maximum calls to serverstop_if_internet_not_available = True Stop if attempting to predict models online and internet is not availableverbose = False Verbose output on communicationsprovider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = prediction *ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 5 Time in seconds between job status checksmax_tries = 20 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = *small medium large Size of job (small, medium, large)requires_gpu = False Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose outputguiGUI-specific parameter required for output

directoryoutput_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- ai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True analysis_mode = *standard agent_session advice_preprocessing directive_extraction failure_diagnosis Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), advice_preprocessing (process user advice into structured format), directive_extraction (extract structured directives from advice), or failure_diagnosis (LLM diagnosis of a diagnosable-terminal error)include_llm_assessment = True For agent_session mode, whether to include LLM-generated assessmentsession_json = None For agent_session mode, the session data as JSON stringraw_advice = None For advice_preprocessing mode, the combined raw advice stringexperiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)file_list_hint = None For advice_preprocessing mode, comma-separated input file namesuser_advice_for_directives = None For directive_extraction mode, the processed user advice stringfailure_error_type = None For failure_diagnosis mode: error type key from diagnosable_errors.yamlfailure_error_text = None For failure_diagnosis mode: encoded error excerpt from the failing programfailure_program = None For failure_diagnosis mode: the PHENIX program that failedfailure_log_tail = None For failure_diagnosis mode: encoded last section of the failing program log
- log_file = None
- log_as_simple_string = None
- file_list_as_simple_string = None
- display_text_as_simple_string = None
- summary_as_simple_string = None
- analysis_as_simple_string = None
- load_existing_analysis = True
- program_name = None
- analysis_file_name = None
- summary_file_name = None
- write_files = True
- timeout = 180
- display_results = True
- analysis_mode = *standard agent_session advice_preprocessing directive_extraction failure_diagnosis Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), advice_preprocessing (process user advice into structured format), directive_extraction (extract structured directives from advice), or failure_diagnosis (LLM diagnosis of a diagnosable-terminal error)
- include_llm_assessment = True For agent_session mode, whether to include LLM-generated assessment
- session_json = None For agent_session mode, the session data as JSON string
- raw_advice = None For advice_preprocessing mode, the combined raw advice string

• experiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)

• file_list_hint = None For advice_preprocessing mode, comma-separated input file names

• user_advice_for_directives = None For directive_extraction mode, the processed user advice string

• failure_error_type = None For failure_diagnosis mode: error type key from diagnosable_errors.yaml

• failure_error_text = None For failure_diagnosis mode: encoded error excerpt from the failing program

• failure_program = None For failure_diagnosis mode: the PHENIX program that failed

• failure_log_tail = None For failure_diagnosis mode: encoded last section of the failing program log

• communicationrun_on_server = True Run job on Phenix serverwait_for_server = False If server is busy or down, wait up to max_wait_timeupdate_wait_time_if_down = 5 Time to wait before trying again on server if it is downupdate_wait_time_if_busy = 5 Time to wait before trying again on server if it is busymax_wait_time = 450 Maximum wait timemax_server_tries = 1 Maximum calls to serverstop_if_internet_not_available = True Stop if attempting to predict models online and internet is not availableverbose = False Verbose output on communicationsprovider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• run_on_server = True Run job on Phenix server

• wait_for_server = False If server is busy or down, wait up to max_wait_time

• update_wait_time_if_down = 5 Time to wait before trying again on server if it is down

• update_wait_time_if_busy = 5 Time to wait before trying again on server if it is busy

• max_wait_time = 450 Maximum wait time

• max_server_tries = 1 Maximum calls to server

• stop_if_internet_not_available = True Stop if attempting to predict models online and internet is not available

• verbose = False Verbose output on communications

• provider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = prediction *ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 5 Time in seconds between job status checksmax_tries = 20 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = *small medium large Size of job (small, medium, large)requires_gpu = False Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose output

• url = None The URL for the Phenix REST server Normally set automatically

• url_type = prediction *ai Type of server url (prediction or ai). Normally set automatically

• port = None The port for interacting with the Phenix REST server Normally set automatically

• token = None Authentication token for accessing the Phenix REST server. Normally set automatically

• timeout = 5 Time to keep trying to connect to a server

• quick_check_interval = 1 Time in seconds between job status checks

- check_interval = 5 Time in seconds between job status checks
- max_tries = 20 Number of tries to get result from server
- max_tries_on_availability = 2 Number of tries to see if server is up
- stop_if_server_not_available = True Stop if server is not available (wrong url or down)
- job_size = *small medium large Size of job (small, medium, large)
- requires_gpu = False Job requires GPU
- running_server_test = False This is a test job
- verbose = False Verbose output
- guiGUI-specific parameter required for output directoryoutput_dir = None
- output_dir = None

AI-Powered Phenix & CCTBX Programming

developer.html

Why use AI: The Phenix developer AI chatbot has all the Phenix and CCTBX documentation and code in its sources. It can help you get going quickly. It can even write unit tests for you.

Be aware: AI chatbots usually do not get everything right on the first try. It is your job to tell them what is wrong (for starters, paste in a traceback!) and to make your code work just the way you want it to.

Be safe: Look up all packages before you install them with pip to make sure they are real. Read the code from AI scripts before running them. Make sure they make sense and do not do anything bad.

Use the right AI tool for the job: Use the Phenix developer chat to write Phenix and CCTBX code. It knows the code base and how to use the Program Template. Use Gemini or other general-purpose AI chatbots to write scripts that do not use Phenix or CCTBX tools or to write HTML. These chatbots are better at programming but they do not know about the Phenix code base.

Chatbots

Phenix developer chat Private, do not share link

Use this for Phenix or CCTBX programming questions or to write Python scripts in Phenix/CCTBX. Designed to allow easy creation of ProgramTemplate files.

CCTBX developer chat

Use this for CCTBX programming questions or scripts.

Phenix user help chat

Use this for questions on how to run Phenix or for general questions about crystallography and cryo-EM structure determination.

Hints on using AI programming chatbots, adding docstrings with AI, and on using the Phenix user chatbot

Programming Documentation

CCTBX developer documentation

Comprehensive guide to using CCTBX tools. High-level cctbx managers, maps, and low-level cctbx objects and tools. Includes tutorials, sample scripts, how to test your code, and many helpful hints on using CCTBX tools.

Phenix developer documentation

How to use the Program Template, how to use the model-building tools in Phenix, handy restraint jiffies, Phenix/CCTBX high-level reference guide.

PHENIX/CCTBX Programming API

Fully indexed documentation of all CCTBX and Phenix Python classes and methods. Includes all code from CCTBX and from Phenix. You can update this on your machine with the commands:

```
phenix.rebuild_phenix_api
```

CCTBX Programming API

Fully indexed documentation and code for all CCTBX Python classes and methods. You can update this on your machine with the commands:

```
phenix.rebuild_cctbx_api
```

Quantum Interface (QI)

quantum_interface.html

Contents

Authors

Nigel W. Moriarty, Dorothee Liebschner

Introduction

Quantum Mechanics (QM) has been shown to provide accurate geometries and energies for a wide range of chemical entities. Using QM for restraints generation is not new, however, using QM in situ restraints generation directly in phenix.refine without additional (and possibibly costly licensed) software is new. The inclusion of MOPAC in the python3 installations of Phenix has made this possible. The Quantum Interface (QI) is a new module that allows the use of MOPAC or a 3rd party QM package – Orca – for a number of procedures.

Various uses

Calculate ligand restraints in situ using Quantum Mechanical Restraints (QMR)

Calculate ligand strain energies using Quantum Mechanical Energies (QME)

Determination of best Histidine tautamer Quantum Mechanical Flipping (QMF)

Tips and Tricks

Model completeness

One of the most important points about Quantum Chemistry is that it is an all-electron method. This means that all atoms (and electrons) need to be specified in order to perform the correct calculation. This is a case where GIGO will really bite you. In standard refinement, missing atoms are not critical but in QM missing atoms are catastrophic.

The QI module calculates the charge based on the atoms in the input model (and the selection) but care must be taken to ensure that the input model is correct down to the protonation. Besides calculating the incorrect result, the QM calculation can have difficulty converging thus resulting in excessive run times.

Timings

QM calculations scale as a power of the number of atoms. This means that increasing the buffer can greatly affect the timings.

Using the parallel features in MOPAC and Orca are generally not linear so the QI module has a feature to prefer simple parallelism on based on jobs that can be useful for some situations.

Literature

In situ ligand restraints from quantum-mechanical methods. D. Liebschner, N.W. Moriarty, B.K. Poon, and P.D. Adams. Acta Cryst. D79, 100-110 (2023).

• In situ ligand restraints from quantum-mechanical methods. D. Liebschner, N.W. Moriarty, B.K. Poon, and P.D. Adams. Acta Cryst. D79, 100-110 (2023).

Using the PHENIX Wizards

running-wizards.html

Purpose

Any Wizard can be run from the PHENIX GUI, from the command-line, and from keyworded parameters files. All three versions are identical except in the way that they take commands and keywords from the user.

This page describes how to run a Wizard and what a Wizard does in general. The specific Wizard help pages describe the details of each PHENIX Wizard.

Overview of Structure Determination with the PHENIX Wizards

You can use the AutoSol Wizard to solve structures by SAD, MAD, SIR/SIRAS, and MIR/MIRAS. The AutoSol Wizards together can carry out MRSAD. The AutoSol Wizard can also combine SAD, MAD, SIR, and MIR datasets and solve the structure using all available data.

Once you have experimental or MR phases, you can carry out iterative model-building, density modification, and refinement with the AutoBuild Wizard to improve your model. Finally you can use the rebuild_in_place feature of the AutoBuild Wizard to make one very good final model.

If your structure contains ligands, you can place them using the LigandFit Wizard

This help page describes how to run the Wizards from a GUI, the command-line, or a parameters file. The individual Wizard documentation pages describe the strategies and commands for each Wizard:

- Automated Structure Solution using AutoSol
- Automated Model Building and Rebuilding using AutoBuild
- Automated Ligand Fitting using LigandFit

Usage

Wizard data directories, sub-directories, Facts, and the PDS (Project Data Storage)

- The directory that you are in when you start up PHENIX is your working directory.
- Each run of a Wizard will have all output data in a subdirectory of your working directory named like this (for AutoSol run 3):

```
AutoSol_run_3_/
```

- This subdirectory will have one or more temporary directories:

```
AutoSol_run_3_/TEMP0/
```

which contain intermediate files. These temporary directories will be deleted when the Wizard is finished (unless you set the parameter clean_up to False)

- For OMIT and MULTIPLE-MODEL runs, the final OMIT maps and multiple models will be in a subdirectory of your run directory:

```
AutoSol_run_3_/OMIT/
AutoSol_run_3_/MULTIPLE_MODELS/
```

- All the parameter values as well as any other information that a Wizard generates during its run is stored in the PDS (Project Data Storage) and/or the Wizard Facts. The Facts are values of parameters and pointers to files in the PDS. The Facts keep track of the current knowledge available to the Wizard. Each time a step is completed by a Wizard, the new Facts are saved (overwriting old ones for that run). As the Facts define the state of the Wizard, the Wizard can be restarted any time by loading the appropriate set of Facts.
- The PDS (Project data storage) will be in your working directory:

```
./PDS/
```

The PDS contains the output of each of your runs for all Wizards and a record of all the Facts (parameters and data) for each run. If you delete a run using the PHENIX Wizard GUI or with a command like "phenix.autosol delete_runs=2", the corresponding entries in the PDS are also deleted. You can copy the PDS from one place to another. Note that if you delete directories such as "AutoSol_run_1_" by hand then the corresponding information remains in the PDS. For this reason it is best to use the GUI or specific commands to delete runs.

Running a Wizard using a multiprocessor machine or on a cluster

You can take advantage of having a multiprocessor machine or a cluster when running the wizards (Currently this applies to the LigandFit and AutoBuild Wizards). For example, adding command

```
nproc=4
```

to a command-line command for a Wizard will use 4 processors to run the wizard (if possible). Normally you will run the parallel processes in the background with the default of

```
background=True
```

If you have a cluster with a batch queue, you can send subprocesses to the batch queue with

```
run_command=qsub
```

(or whatever your batch command is). In this case you will use

```
background=False
```

so that the batch queue can keep track of your jobs.

The Wizards divide the data into nbatch batches during processing. The value of

```
nbatch=3
```

is set from 3 to 5 by default (depending on the Wizard) and is appropriate if you have up to nbatch processors. If you have more, then you may wish to increase nbatch to match the number of processors. The reason it is done this way is that the value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch.

Running a Wizard from a GUI

Basic operation of a Wizard from the GUI

- Start up the PHENIX GUI in your working directory by typing "phenix"
- Answer "yes" to the question "Do you want to make it a project directory?".

• Launch a Wizard from the PHENIX GUI by double-clicking on the name of the Wizard ("AutoSol") under "Wizards" in the Strategy Interface of the main GUI.

• The Wizard will come up in a blue window and will open a grey Parameters window asking you for information on what files to use and what to do.

• Enter the file names and make choices as necessary (NOTE: to select a file click on the yellow box to the right of the file entry field. To add a new file entry field click on the "Parameter group options" tab if present).

• Proceed to the next window by clicking "Continue" in the upper left corner of the grey Parameters window.

• The Wizard will guide you through the necessary inputs, then it will continue on its own until it is finished.

• When the Wizard is done, you can double-click on the Display icon (the little magnifying glass on the upper left of the blue Wizard window) to show a list of files and maps that can be displayed. (NOTE: The Display Options window is updated when you open it. Once this window is open you cannot open it again until you close it. Sometimes this window may be behind other windows and this will prevent you from opening it again.)

• You can open the Parameters window any time the Wizard is stopped by clicking on the Parameters icon (4 little lines in the upper left corner of the blue Wizard window). This allows you to carry out some of the more advanced options below.

• Your output log file will be in a file called "AutoSol.1.output" for an AutoSol run. You can also see the same file by clicking on the "LOG" button at the lower right of the blue or green window.

Keeping track of multiple runs of a Wizard from the GUI

• You can run more than one Wizard job at a time if you want. Each run of a Wizard is put in a separate sub-directory (e.g., "AutoSol_run_1_").

• When you start a Wizard, it will start a new run of that Wizard.

• If you want to continue on with the highest-numbered run of a Wizard, you can start the Wizard with the continue button for that Wizard (for example the continue_AutoSol button).

• If you want to go back to a previous run, you can use the Run Control and Run Number selections near the bottom of any Parameters window (NOTE: to open the parameters window click on the lines at the upper left of the blue Wizard window). Select goto_run and choose a run number to go to.

• If you want to copy a previous run and go on, use the Run Control and Run Number selections and select copy_run and choose a run number to copy. The Wizard will create a new run (with number equal to the highest previous number plus one) and carry on with it.

• To see what runs are available, select View or Delete Runs in the Navigate tab at the lower left of any Parameters window.

• If you want to stop the Wizard, hit the PAUSE button on the green Wizard window (the Wizard is green when running, blue or purple when stopped). NOTE: this may take a little time, particularly if Phaser or HYSS or phenix.refine are running. In those cases if you really want to stop the Wizard right away, got to "Strategy" and then select "Stop Strategy" and it will be stopped.

Setting parameters of a Wizard from the GUI

- You can set any parameter in a Wizard by selecting the variable in the Choose Variable to Set tab. The next time you click Continue, the Wizard will save all the current inputs as usual, and then instead of going on to the next step, it will open a window asking you for the new value of that variable. When you enter it and press Continue, the Wizard will continue on with what it was doing, but with this new value.
- NOTE that some parameters (e.g., resolution) may affect many steps. If a prior step is affected by a parameter that is changed, the Wizard does not go back and change it. If you want the parameter change to affect something that has already been done, you need to re-run the corresponding step.
- NOTE that you can set any SOLVE, RESOLVE or RESOLVE_PATTERN keyword when you are running a Wizard using the "resolve_command", "solve_command" or "resolve_pattern_command" keywords. These can be set in the GUI from the Choose Variable pull-down menu. You just type in the command to the entry form like this: (for resolve_command):

```
res_start 4.0
```

telling resolve in this case to start out density modification at a resolution of 4 A. This allows you to control what solve, resolve and resolve_pattern do more finely than you otherwise can in the Wizards.

Navigating steps in a Wizard from the GUI

- When the Wizard is done or Paused, you can select any available step in the Navigate tab at the middle bottom of any Parameter window. This tells the Wizard to get any necessary inputs for that step and to then carry it out.
- The Wizards normally start out in Manual mode (one step at a time, asking user for inputs). Once the necessary inputs are entered, the Wizard enters Automatic mode (no more asking for inputs until something required is missing). You can control this by specifying Manual or Automatic in the Auto/Manual tab at the bottom right of any Wizard.

Running a Wizard from the command-line

Basic operation of a Wizard from the command-line

- You can run a wizard from the command line like this (autosol is the AutoSol wizard):

```
phenix.autosol data=w1.sca seq_file=seq.dat 2 Se
```

- The command_line interpreter will try to interpret obvious information (2 means sites=2, Se means atom_type=Se) and will run the wizard.
- To see all the information about this wizard and the keywords that you can set for this wizard, type:

```
phenix.autosol --help all
```

- Any wizard keyword can be entered at the command line (not just the ones labelled "command-line only"). The documentation for each wizard lists all the keywords that apply to that wizard.
- If you want to stop a Wizard, you can create a file "STOPWIZARD" and put it in the subdirectory (i.e., AutoSol_2_/) where the Wizard is running. This is like hitting the PAUSE button on the GUI and stops the wizard cleanly.

Keeping track of multiple runs of a Wizard from the command-line

- When you start a Wizard from the command line, the default is to start a new run of that Wizard.
- To see all the available runs of this Wizard, type:

```
phenix.autosol show_runs
```

- To delete runs 1,2 and 4-7 of this Wizard, type something like this:

```
phenix.autosol delete_runs="1 2 4-7"
```

Note that the group of numbers is enclosed in quotes ("). This tells the input parser (iotbx.phil) that all these numbers go with the one keyword of delete_runs. Note also that there are no spaces around the "=" sign!

- To go back to run 2 and carry on (remembering all previous inputs and possibly adding new ones, in this case setting the resolution) type something like:

```
phenix.autosol run=2 resolution=3.0
```

- To carry on with the current highest-numbered run (remembering all previous inputs and possibly adding new ones, in this case setting the resolution) type something like:

```
phenix.autosol carry_on resolution=3.0
```

- To copy run 2 to a new run and carry on from there (remembering all previous inputs and possibly adding new ones, in this case setting the resolution) type something like:

```
phenix.autosol copy_run=2 resolution=3.0
```

Setting parameters of a Wizard from the command-line

When you run a Wizard from the command-line, two files are produced and put in the subdirectory of the Wizard (e.g., AutoBuild_run_3_/).

- A parameters (".eff") file will be produced that you can edit to rerun the Wizard:

```
phenix.autosol autosol.eff
```

This autosol.eff file (for AutoSol) contains the values of all the AutoSol parameters at the time of starting the Wizard.

Note that the syntax in the autosol.eff file is very slightly different than the syntax from the command line. From the command line, if a value has several parts, you enclose them in quotes and there are no spaces around the "=" sign:

```
phenix.autosol ... input_phase_labels="FP PHIM FOMM"
```

In the .eff file, you MUST leave off the quotes or the three values will be treated as one, and you should leave blanks around the "=" sign:

```
input_phase_labels = FP PHIM FOMM
```

The reason these are different is that in the .eff file, the structure of the file and the brackets tell the PHIL parser what is grouped together, while from the commmand line, the quotes tell the parser what is to be grouped together.

- A parameters file (".eff") is produced that you can edit and use like this:

```
phenix.autosol parameters.eff
```

- To get keyword help on a specific keyword you can type:

```
phenix.autosol --help data # get help on the keyword data for autosol
```

- To show current Facts (values of all parameters) for highest_numbered run:

```
phenix.autosol show_facts
```

- To show current Facts (values of all parameters) for run 3:

```
phenix.autosol run=3 show_facts
```

- To show current summary:

```
phenix.autosol show_summary
```

- When you use a keyword like data= you need to give enough information to specify this keyword uniquely. You can see all the keywords for each PHENIX Wizard or tool at the end of the documentation for that Wizard or tool. This will have entries like this (for AutoSol):

```
autosol
 sites= None Number of heavy-atom sites. (Command-line only)
```

which describes the keyword sites in the scope defined by autosol. You can explicitly specify this on the command line with:

```
autosol.sites=3
```

which in this case is entirely the same as

```
sites=3
```

- NOTE that you can set any SOLVE, RESOLVE or RESOLVE_PATTERN keyword in PHENIX using the "resolve_command", "solve_command" or "resolve_pattern_command" keywords from the command line. The format is a little tricky: you have to put two sets of quotes around the command like this:

```
resolve_command="'ligand_start start.pdb'" # NOTE ' and " quotes
```

This will put the text

```
ligand_start start.pdb
```

at the end of every temporary command file created to run resolve.

Running a Wizard from a parameters file

Parameters files are an easy way to specify any parameters that you want to use when running a Wizard. They are structured in a clear way and you can edit them to set the values that you want.

You can get a parameters file to edit with any wizard by running the wizard with the flag "--show_defaults":

```
phenix.autosol --show_defaults
```

Here is a parameters file "sad.eff" to run a SAD dataset with "phenix.autosol sad.eff":

```
autosol { atom_type = Se
 sites = 2
 seq_file = sequence.dat
 crystal_info {
 space_group = C2
 unit_cell = 76.08 27.97 42.36 90 103.2 90
 resolution = 2.6
 }
 wavelength {
 data = high.sca
 lambda = 0.9600
 f_prime = -1.5
 f_double_prime = 3
 }
}
```

Note the scope names ("autosol" or "crystal_info") followed by paired brackets ({ ....}) which enclose sets of parameters that are related.

The values of parameters are usually entered on one line, without quotation marks (as in this example) unless they are to be all considered as a single item.

You can specify almost anything either in a parameters file or on the command line. In the above example you could also just say:

```
phenix.autosol atom_type = Se sites = 2 seq_file = sequence.dat \
 space_group = C2 \
 unit_cell = "76.08 27.97 42.36 90 103.2 90" \
 resolution = 2.6 \
 data = high.sca \
 lambda = 0.9600 \
 f_prime = -1.5 \
 f_double_prime = 3
```

(note that the cell parameters are in quotes on the command line and not in the parameters file) and you would get the same results. For simple cases the command-line format is fine, but for anything with a lot of parameters to set it is much easier to just edit a parameters file.

Specific limitations and problems:

- In the GUI version of Wizards, The Display Options window is updated only when you open it. Further, once this window is open you cannot open it again until you close it. Sometimes this window may be behind other windows and this will prevent you from opening it again until you close the open window.
- The Wizards use file names based on the names of your input files, but they do not differentiate between files with the same name coming from different directories. Consequently you should not use two files with different contents but with the same file name as inputs to a Wizard, even if they come from separate starting directories.
- If you stop a Wizard and continue on with a command such as phenix.autobuild run=2 then you can change most parameters with keywords just as if you were starting from scratch, but if you had previously changed a keyword away from the default, you cannot set it back to the default in this way (the Wizard ignores keywords that are the same as the default).
- You should not work on the same run in two ways at the same time. This can lead to unpredictable results because the two runs will really be the same run and the data and databases for the two runs will be overwriting each other. This means you need to be careful that if you goto_run 1 of a Wizard in one window that you do not also goto_run 1 of the same Wizard in another window. On the other hand, it is perfectly fine to work on run 1 of a Wizard in one window and run 2 of the same Wizard in another window.
- The PHENIX Wizards can take most settings of most space groups, however they can only use the hexagonal setting of rhombohedral space groups (eg., #146 R3:H or #155 R32:H), and cannot use space groups 114-119 (not found in macromolecular crystallography) even in the standard setting due to difficulties with the use of asuset in the version of ccp4 libraries used in PHENIX for these settings and space groups.

Literature

Additional information

The Phenix atom selection syntax

atom_selections.html

Contents

Introduction

Many of the programs in Phenix, and phenix.refine in particular, allow (or require) you to specify selections of atoms in a model. Common examples include:

TLS (constrained anisotropic displacement) and rigid-body refinement groups Selections for isotropic versus anisotropic B-factor refinement Atoms to leave out for omit map calculation Subset of atoms to modify properties for (in PDBTools) Harmonically restrain atoms during refinement

- TLS (constrained anisotropic displacement) and rigid-body refinement groups
- Selections for isotropic versus anisotropic B-factor refinement
- Atoms to leave out for omit map calculation
- Subset of atoms to modify properties for (in PDBTools)
- Harmonically restrain atoms during refinement

All of these parameters are defined using a common syntax similar to what is used in CNS or PyMOL. The syntax allows for selection of atomic properties such as atom name, residue number, chain ID, or B-factor, which can be combined by the use of simple boolean statements (and, or, or not).

Video Tutorial

The tutorial video is available on the Phenix YouTube channel and covers the following topics:

- Basic overview
- Examples for selection syntax
- Examples for combined expressions
- How to test if an expression works (phenix.pdb_atom_selection)

Examples for selection expressions

All atoms

```
all
```

All C-alpha atoms (not case sensitive)

```
name ca
```

All atoms with H in the name (* is a wildcard character)

```
name *H*
```

Atoms names with * (backslash disables wildcard function)

```
name o2\*
```

All atoms in 'A' conformation

```
altloc A
```

Atom names with spaces

```
name 'O 1'
```

Atom names with primes don't necessarily have to be quoted

```
name o2'
```

Boolean "and", "or", and "not"

```
resname ALA and (name ca or name c or name n or name o)
chain a and not altid b
resid 120 and icode c and model 2
segid a and element c and charge 2+ and anisou
```

Note that the and, or, and not operators have equal priority, so parentheses may be required to indicate which clauses go together. The first example above selects for all atoms with the specified names in alanine residues, but if the parentheses are omitted:

```
resname ALA and name ca or name c or name n or name o
```

it will instead select C-alpha atoms in ALA, plus all atoms named C, N, or O regardless of residue name.

Also note that the outcome of boolean expression can be somewhat different from what one may expect comparing it with natural language. For example, if one wants to select ALA and GLY residues in the molecule, the selection should be

```
resname ALA or resname GLY
```

because suitable residues have residue name ALA or GLY. On the contrary, selection

```
resname ALA and resname GLY
```

will result in empty selection because there is no residue that can satisfy both conditions simultaneously.

A single residue by number

```
resseq 188
```

Note that if there are several chains containing residue number 188, all of them will be selected. To be more specific and select residue 188 in particular chain:

```
chain A and resid 188
```

this will select residue 188 only in chain A.

A range of residues

```
resseq 2:10
```

This selects all residue numbers falling within this range, independent of their order in the PDB file. The numbering must be ascending; resseq 10:2 will not work.

Insertion codes

resid combines the residue number (resseq) with the insertion code (icode); thus the following selections are identical:

```
resid 188
resseq 188 and icode ' '
```

as are these:

```
resid 27C
resseq 27 and icode 'C'
```

Specifying ordered ranges of residues

For some purposes, such as specifying TLS groups, it may be impractical to specify a numerical range of resseq attributes. This is especially the case when the residue numbering is not continuous and ascending, which is found in some PDB entries, or when the insertion code is non-blank. An alternative is to define a series of residues by their combined number and insertion code, using the through keyword:

```
chain A and resid 1000 through 17J
```

In this case, the residues will be examined in the order that they appear in the PDB file, and every atom falling between 1000 and 17J in chain A will be included. This is the only situation where the ordering of atoms is important in atom selections.

B-factors and occupancies

You may also select atoms based on their numerical properties:

```
bfactor &gt; 80
bfactor = 0
occupancy &lt; 1
occupancy = 0
```

Other

A few additional keywords are supported:

```
element H
water
pepnames
hetero
```

These examples specify hydrogen atoms, water molecules (detected by residue name), common amino acid residues, and atoms labeled as HETATM, respectively.

Selecting no atoms

```
(not all)
```

The default value of most selections is None (which in the GUI will appear as a blank field), but the treatment of this differs depending on how the atom selection will be used. For omit selections, None is equivalent to (not all); for rigid-body selections, phenix.refine will default to treating the entire model as a single rigid body. For B-factor refinement, leaving the selections set to None will result in their parameterization being determined based on the input model and atom type (hydrogens are treated specially).

Graphical selection

In the phenix.refine GUI, any valid atom selection can be visualized if you have a suitable graphics card and have already loaded a PDB file with valid symmetry information. The graphics window can be opened by clicking the "View/pick" button next to any atom selection field. Depending on the size of your structure, it may take several seconds for PHENIX to determine the atomic connectivity. The current selection, if any, will be highlighted:

The View/pick button opens a window that allows you to type in selections and immediately visualize the results, including the number of atoms selected. On mice with at least two buttons, clicking on an atom with the right button will open a menu for selecting residue ranges or chains. However, we recommend that you learn the selection syntax, as it is much more flexible than mouse controls. Selections made in the graphics window will be sent automatically to the appropriate control.

Once this window is open, it does not need to be closed; clicking a different "View/pick" button will transfer control of the display. (On Linux, you will first need to open the Actions menu and click Restore parent window to transfer mouse and keyboard controls back to the configuration dialog.)

"Smart" selections

In some contexts (primarily in phenix.refine and PDBTools), the geometry restraints information is used to extend the allowed keywords:

```
resname ALA and backbone
resname ALA and sidechain
peptide backbone
rna backbone or dna backbone
water or nucleotide
dna and not (phosphate or ribose)
within(5, (nucleotide or peptide) backbone)
```

Note however that these options cannot be visually selected in the Phenix GUI at present, and they may not be recognized by all programs.